Problem 1 (4 pts)

Consider the following initial value problem:

$$\begin{cases} x'(t) = f(t, x(t)) & \text{on } [0,1] \\ x(0) = \alpha \end{cases} \tag{1}$$

and the following general difference equation to approximate the solution to the IVP above:

$$\begin{cases} w_{i+1} = w_i + h\Phi(t_i, w_i, h) \\ w_0 = \alpha \end{cases} \tag{2}$$

(a) Recall the Taylor series method of second order is defined with

$$\Phi(t, w, h) := f(t, w) + \frac{h}{2}(f_t(t, w) + f_w(t, w)f(t, w)).$$

Find an expression of b, α_1, β_1 such that

$$bf(t + \alpha_1, w + \beta_1) = f(t, w) + \frac{h}{2}(f_t(t, w) + f_w(t, w)f(t, w)) + O(h^2)$$

(b) Suggest a Runge-Kutta method based on the above computation.

(c) Write down the Butcher Tableau of the above Runge-Kutta method.

(a) Expanding $bf(t + \alpha_1, w + \beta_1)$ we get,

$$bf(t + \alpha_1, w + \beta_1) = bf(t, w) + b\alpha_1 f_t(t, w) + b\beta_1 f_w(t, w) + O(h^2)$$

Comparing with the Taylor series method of second order we see that,

$$\begin{cases} b = 1 \\ b\alpha_1 = h/2 \\ b\beta_1 = \frac{h}{2}f(t, w) \end{cases} \implies \begin{cases} b = 1 \\ \alpha_1 = h/2 \\ \beta_1 = \frac{h}{2}f(t, w) \end{cases}$$

(b) From part (a) we have that $\Phi(t, w; h) \approx f(t + \frac{h}{2}, w + \frac{h}{2}f(t, w))$ thus,

$$\begin{cases} w_{i+1} = w_i + hf(t_i + \frac{h}{2}, w_i + \frac{h}{2}f(t_i, w_i)) \\ w_0 = \alpha \end{cases}$$

From here we see that this is actually the Midpoint method, which falls into the class of Runge-Kutta

methods. Therefore the suggestions is to use the Midpoint method.

(c) Note there are two stages and we have, $\vec{c} = \begin{bmatrix} 0 \\ 1/2 \end{bmatrix}$, $\vec{b} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, and $A = \begin{bmatrix} 0 & 0 \\ 1/2 & 0 \end{bmatrix}$. Hence we get the following Butcher Tableau

$$\begin{array}{c|c} \vec{c} & A \\ \hline & \vec{b}^{\mathsf{T}} \end{array} = \begin{array}{c|cc} 0 & 0 & 0 \\ 1/2 & 1/2 & 0 \\ \hline & 0 & 1 \end{array}.$$

Problem 2 (4 pts)

Consider the following initial value problem:

$$\begin{cases} x'(t) = f(t, x(t)) & \text{on } [0,1] \\ x(0) = x_0 \end{cases}$$

Consider the following fourth-order Runge-Kutta method (RK4):

$$w_{i+1} = w_i + \frac{1}{6}(F_1 + 4F_3 + F_4)$$

where

$$\begin{cases} F_1 = hf(t_i, w_i) \\ F_2 = hf(t_i + \frac{1}{2}h, w_i + \frac{1}{2}F_1) \\ F_3 = hf(t_i + \frac{1}{2}h, w_i + \frac{1}{4}F_1 + \frac{1}{4}F_2) \\ F_4 = hf(t_i + h, w_i - F_2 + 2F_3) \end{cases}$$

(a) Write down the Butcher Tableau of the above fourth-order Runge-Kutta method (RK4).

(b) Is this scheme implicit or explicit?

(c) Show the consistency of this Runge-Kutta method by applying a theorem.

(a) Let's first obtain the weights of the F_i the b_i's. By inspection we see $\vec{b}^{\mathsf{T}} = (\frac{1}{6}, 0, \frac{4}{6}, \frac{1}{6})$. Now by inspection we can also find the time increments, c_i. We see $\vec{c}^{\mathsf{T}} = (0, \frac{1}{2}, \frac{1}{2}, 1)$. Finally we get a_{ij} by inspecting each F_i and the weights of F_i that are being added to w_i. We get,

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 1/2 & 0 & 0 & 0 \\ 1/4 & 1/4 & 0 & 0 \\ 0 & -1 & 2 & 0 \end{bmatrix}.$$

Therefore we get the following butcher Tableau

$$
\begin{array}{c|cccc}
0 & 0 & 0 & 0 & 0 \\
1/2 & 1/2 & 0 & 0 & 0 \\
1/2 & 1/4 & 1/4 & 0 & 0 \\
1 & 0 & -1 & 2 & 0 \\
\hline
 & 1/6 & 0 & 4/6 & 1/6
\end{array}
$$

(b) This scheme is explicit since the $(a)_{ij}$ matrix is lower triangular with a 0 diagonal.

(c) This RK4 implementation is consistent since $\sum_{i=1}^{4} b_i = 1/6 + 0 + 4/6 + 1/6 = 1$. The method is additionally stable since $c_i = \sum_{j=1}^{4} a_{ij}$ for $i = 1, 2, 3, 4$.

Problem 3 (4 pts)

Consider the following initial value problem:

$$
\begin{cases} x'(t) = f(t, x(t)) & \text{on } [0, 1] \\ x(0) = x_0 \end{cases}
$$

Write it in integral form:

$$
\begin{cases} x(t+2h) - x(t) = \int_t^{t+2h} f(s, x(s))\, \mathrm{d}s \\ x(0) = x_0 \end{cases} \tag{*}
$$

where h is a step-size. Let the function $G(t) = f(t, x(t))$ be smooth and let $t_i = ih$, $x_i = x(t_i)$.

(a) Let $P(t)$ be the Lagrange interpolation polynomial of $G(t)$ at the nodes $t = t_i, t_{i+1}, t_{i+2}$ with its values $P(t_{i+k}) = G(t_{i+k}) = f(t_{i+k}, x_{i+k})$ for $k = 0, 1, 2$. Find a formula for the approximation of the following integral

$$
\int_{t_i}^{t_i+2h} f(s, x(s))\, \mathrm{d}s \approx \int_{t_i}^{t_i+2h} P(s)\, \mathrm{d}s \tag{**}
$$

by the integral of the polynomial $P(t)$. (Please notice that we are now assuming that we know the exact values of $x(t)$ at $t = t_i, t_{i+1}, t_{i+2}$, i.e. x_{i+k} for $k = 0, 1, 2$, for some unknown reasons.)

(b) Suggest an implicit multistep scheme from the above quadrature approximation.

(a) First let's compute $P(t)$ using the three given nodes, t_i, t_{i+1}, t_{i+2}. We get,

$$
P(t) = G(t_i)\frac{(t - t_{i+1})(t - t_{i+2})}{(t_i - t_{i+1})(t_i - t_{i+2})} + G(t_{i+1})\frac{(t - t_i)(t - t_{i+2})}{(t_{i+1} - t_i)(t_{i+1} - t_{i+2})} + G(t_{i+2})\frac{(t - t_i)(t - t_{i+1})}{(t_{i+2} - t_i)(t_{i+2} - t_{i+1})}
$$

We can simplify the denominators since time steps are equally spaced we get,

$$P(t) = \frac{G(t_i)}{2h^2}(t - t_{i+1})(t - t_{i+2}) - \frac{G(t_{i+1})}{h^2}(t - t_i)(t - t_{i+2}) + \frac{G(t_{i+2})}{2h^2}(t - t_i)(t - t_{i+1})$$

Now we integrate $\int_{t_i}^{t_{i+2}} P(t)\ \mathrm{d}t$ using the u sub $u = t - t_{i+1}$; $\mathrm{d}u = \mathrm{d}t$. We get,

$$\begin{aligned}
\int_{t_i}^{t_{i+2}} P(t)\ \mathrm{d}t &= \frac{G(t_i)}{2h^2}\int_{-h}^{h} u(u-h)\ \mathrm{d}u - \frac{G(t_{i+1})}{h^2}\int_{-h}^{h}(u+h)(u-h)\ \mathrm{d}u + \frac{G(t_{i+2})}{2h^2}\int_{-h}^{h}(u+h)u\ \mathrm{d}u \\
&= G(t_i)\frac{1}{2h^2}\cdot\frac{2h^3}{3} + G(t_{i+1})\frac{-1}{h^2}\cdot\frac{-4h^3}{3} + G(t_{i+2})\frac{1}{2h^2}\cdot\frac{2h^3}{3} \\
&= \frac{1}{3}hG(t_i) + \frac{4}{3}hG(t_{i+1}) + \frac{1}{3}hG(t_{i+2}) \\
&= h\left[\frac{1}{3}f(t_i, x_i) + \frac{4}{3}f(t_{i+1}, x_{i+1}) + \frac{1}{3}f(t_{i+2}, x_{i+2})\right]
\end{aligned}$$

Using the approximation (**) and (*) we get,

$$\begin{aligned}
x_{i+2} - x_i &\approx h\left[\frac{1}{3}f(t_i, x_i) + \frac{4}{3}f(t_{i+1}, x_{i+1}) + \frac{1}{3}f(t_{i+2}, x_{i+2})\right] \\
\implies x_{i+2} &\approx x_i + h\left[\frac{1}{3}f(t_i, x_i) + \frac{4}{3}f(t_{i+1}, x_{i+1}) + \frac{1}{3}f(t_{i+2}, x_{i+2})\right].
\end{aligned}$$

(b) Given this quadrature rule we suggest the following implicit 2-step method with $a_0 = -1$, $a_1 = 0$, $a_2 = 1$ and $b_0 = \frac{1}{3}$, $b_1 = \frac{4}{3}$, and $b_2 = \frac{1}{3}$. This quadrature rule looks oddly similar to Simpson's rule...

Problem 4 (4 pts)

Consider the following multi-step method:

$$w_{i+2} - w_{i+1} = h\left[\frac{3}{2}f(t_{i+1}, w_{i+1}) - \frac{1}{2}f(t_i, w_i)\right]$$

where $h > 0$ is the step size.

(a) Write down the definition of local truncation error of the above scheme.

(b) Check that the method is consistent by a theorem.

(c) Show the above method is of second order by definition of local truncation error.

(a) This is a 2-step method so $m = 2$. Thus we get,

$$\tau_{i+2}(h) = -\frac{1}{h}w(t_i + h) + \frac{1}{h}w(t_i + 2h) + \frac{1}{2}w'(t_i) - \frac{3}{2}w'(t_i + h)$$

(b) We check consistency by checking if $\sum_{j=0}^{2} a_j = 0$ and $\sum_{j=0}^{m} j a_j = \sum_{j=0}^{m} b_j$. We have $a_0 = 0$,

$a_1 = -1$, $a_2 = 1$, and $b_0 = -1/2$, $b_1 = 3/2$, $b_2 = 0$. Computing yields,

$$\sum_{j=0}^{2} a_j = 0 - 1 + 1 = 0;$$
$$\sum_{j=0}^{2} j a_j = 0 \cdot 0 + 1 \cdot (-1) + 2 \cdot (1) = 1,$$
$$\sum_{j=0}^{2} b_j = -1/2 + 3/2 + 0 = 1.$$

Thus the 2-step method is consistent by the theorem we covered in class.

(c) Consider the local truncation error we computed in part (a). We have,

$$\tau_{i+2}(h) = \underbrace{-\frac{1}{h}w(t_i + h)}_{A} + \underbrace{\frac{1}{h}w(t_i + 2h)}_{B} + \underbrace{\frac{1}{2}w'(t_i)}_{C} \underbrace{-\frac{3}{2}w'(t_i + h)}_{D}$$

Now we use apply a Taylor series expansion for all 4 terms and we get,

$$\begin{array}{lllll}
A: & -\frac{1}{h}w(t_i) & -w'(t_i) & -\frac{h}{2}w''(t_i) & + O(h^2) \\
B: & \frac{1}{h}w(t_i) & +2w'(t_i) & +2hw''(t_i) & + O(h^2) \\
C: & & +\frac{1}{2}w'(t_i) & & \\
D: & & -\frac{3}{2}w'(t_i) & -\frac{3h}{2}w''(t_i) & + O(h^2)
\end{array}$$

Finally note that all the lower order terms cancel and we have $\tau_{i+2}(h) = O(h^2)$ and thus this method is at least of order 2. Computing the next term in the Taylor expansion shows that the h^2 terms do not cancel and thus the method *is* of order 2.

Problem 5 (4 pts)

Computational problem. Consider the following initial value problem:

$$\begin{cases} x'(t) = (x + t)^2 & \text{on } [0, 1] \\ x(0) = -1 \end{cases}$$

(a) Approximate the solution $x(t)$ by the modified Euler method with a mesh size $h = 0.01$. Submit both your code and the graph of the approximate solution.

(b) Approximate the solution $x(t)$ by the classical Fourth-order Runge-Kutta method (RK4)

$$w_{i+1} = w_i + \frac{1}{6}(F_1 + 2F_2 + 2F_3 + F_4)$$

where

$$\begin{cases} F_1 = hf(t_i, w_i) \\ F_2 = hf(t_i + \frac{1}{2}h, w_i + \frac{1}{2}F_1) \\ F_3 = hf(t_i + \frac{1}{2}h, w_i + \frac{1}{2}F_2) \\ F_4 = hf(t_i + h, w_i + F_3) \end{cases}$$

with a mesh size $h = 0.01$. Submit both your code and the graph of the approximate solution.

(a) Below is the graph for the Modified Euler Method Approximate solution as well as the original Euler Method. We note a slight increase in accuracy as the approximate solution for the original method begins to trail off of the analytic solution $(\tan(t - \pi/4) - t)$ around $t = 0.2$, but does much better in the Modified Euler Method. Code is found in the code appendix.

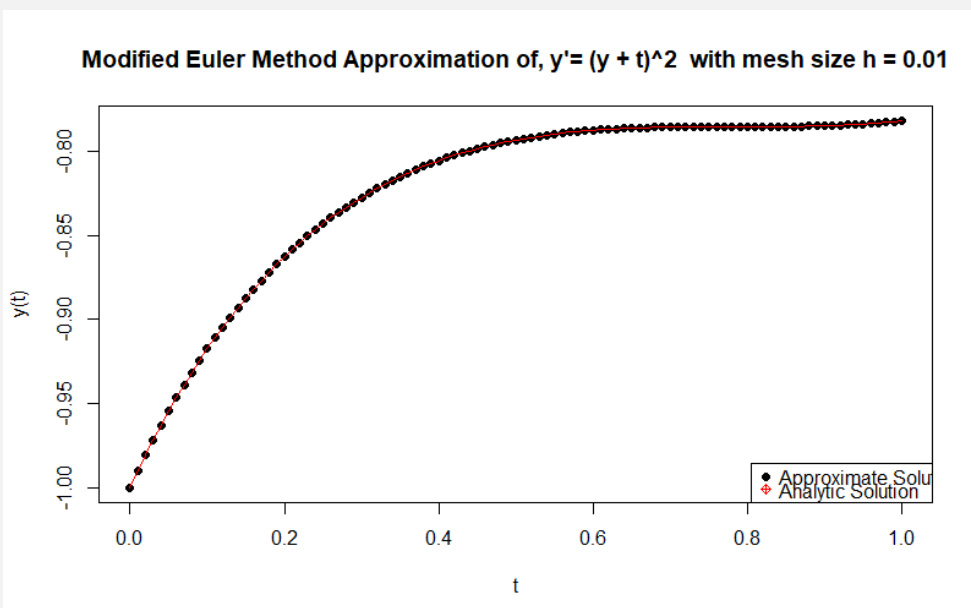

Figure 1: Modified Euler Method

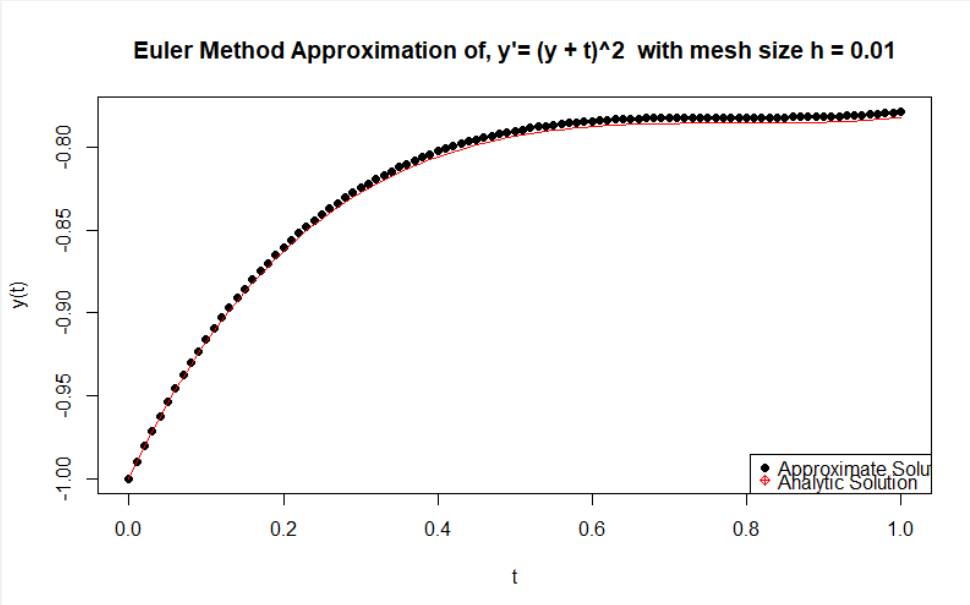

Figure 2: Euler Method

(b) Using the classical RK4 we get the following output of the approximate solution below. We note that the solution is ever so slightly better in agreement with the analytical solution than what we got from the modified Euler method. Code is found in the code appendix.

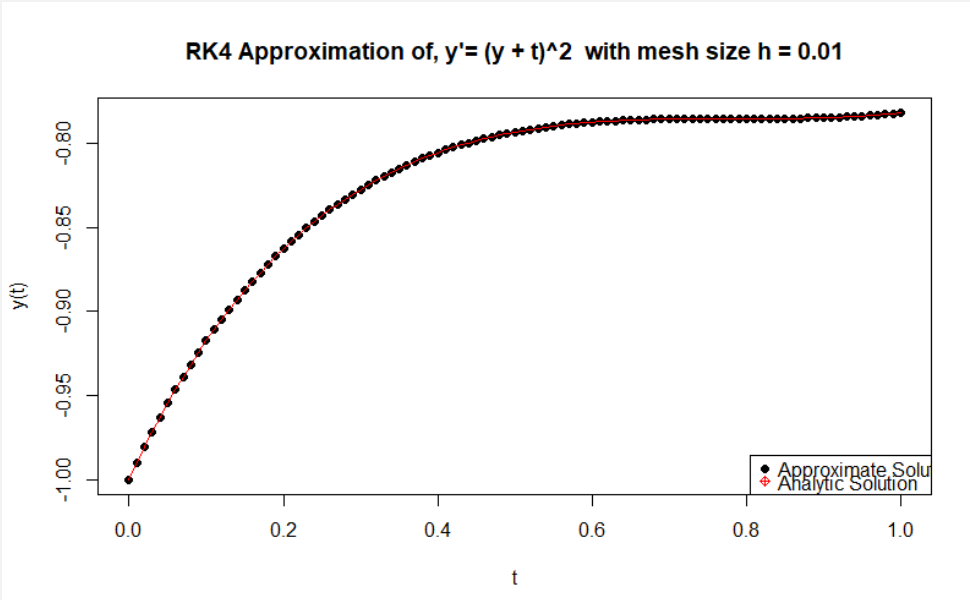

Figure 3: Fourth-order Runge-Kutta Method (RK4)

Code Appendix

Algorithms are written in R.

Modified Euler Method Code

```r
## Modified Euler's Method

## give expression in terms of y and t
mod_euler_method <- function(y, h, t0, y0, a, b){

  y_expr <- substitute(y)

  if (is.symbol(y_expr)) {
    y_expr <- eval.parent(y_expr)
  }

  ## num evalutaion function
  EVAL <- function(f, t, y){
    eval(f, list(t = t, y = y))
  }

  t_f <- seq(t0, b, by = h)
  y_f <- numeric(length(t_f))
  y_f[1] <- y0

  for(i in seq_len(length(t_f) - 1)){
    y_f[i + 1] <- y_f[i] + (h/2) * EVAL(y_expr, t_f[i], y_f[i]) + (h/2) * EVAL(y_expr, t_f[i+1],
     y_f[i] + h*EVAL(y_expr, t_f[i], y_f[i]))
  }

  t_b <- seq(t0, a, by = -h)
  y_b <- numeric(length(t_b))
  y_b[1] <- y0

  for(i in seq_len(length(t_b) - 1)){
    y_b[i + 1] <- y_b[i] - h * EVAL(y_expr, t_b[i], y_b[i])
  }

  t_all <- c(rev(t_b), t_f[-1])
  y_all <- c(rev(y_b), y_f[-1])

  plot(t_all, y_all,
       type="b",
       pch=19,
       xlab="t",
       ylab="y(t)",
       main=paste("Modified Euler Method Approximation of, y'=",as.expression(y_expr),
                  " with mesh size h =",h))
}


mod_euler_method((y+t)^2, 0.01, 0, -1, 0,1)
curve(tan(x-pi/4)-x, add = TRUE, col="red")
legend("bottomright", legend = c("Approximate Solution", "Analytic Solution"),
```

```
       col = c("black", "red"), pch = c(19,10))
```

Runge-Kutta Code

```
## RK4 Code##

## inputs: h, y, yo, a, b
RK4 <- function(h, y, y0, a, b){

  y_expr <- substitute(y)

  if (is.symbol(y_expr)) {
    y_expr <- eval.parent(y_expr)
  }

  ## num eval functions
  y_prime <- function(f, t, y){
    eval(f, list(t = t, y = y))
  }

  ## sequences for t and y
  t_seq <- seq(a, b, by = h)
  n <- length(t_seq)
  y_seq <- numeric(n)
  y_seq[1] <- y0

  ## main update
  for(i in 1:(n-1)){
    ## using RK4 definitions
    F1 <- h*y_prime(y_expr, t_seq[i], y_seq[i])
    F2 <- h*y_prime(y_expr, t_seq[i]+(h/2), y_seq[i]+(1/2)*F1)
    F3 <- h*y_prime(y_expr, t_seq[i]+(h/2), y_seq[i]+(1/2)*F2)
    F4 <- h*y_prime(y_expr, t_seq[i]+h, y_seq[i]+F3)
    ## main explicit update
    y_seq[i + 1] <- y_seq[i] + (1/6)*(F1 + 2* F2 + 2 *F3 + F4)
  }

  ## plotting approximation
  plot(t_seq, y_seq,
       type="b",
       pch=19,
       xlab="t",
       ylab="y(t)",
       main=paste("RK4 Approximation of, y'=",as.expression(y_expr),
                  " with mesh size h =",h))
}

RK4(0.01, (y+t)^2, -1, 0, 1)
curve(tan(x-pi/4)-x, add = TRUE, col="red")
legend("bottomright", legend = c("Approximate Solution", "Analytic Solution"),
       col = c("black", "red"), pch = c(19,10))
```